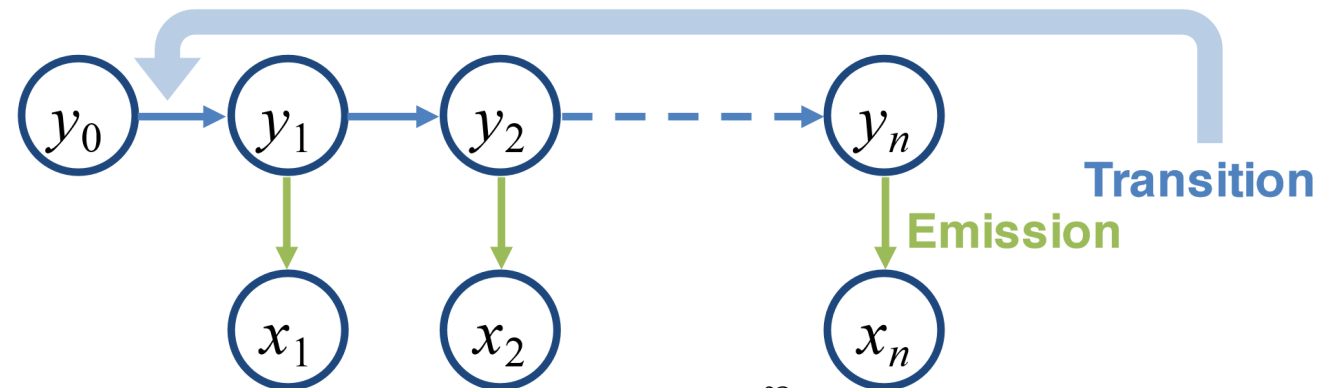


Probabilistic Sequence Models

- Probabilistic sequence models allow integrating uncertainty over multiple, interdependent classifications and collectively determine the most likely global assignment
- Classic solution: Hidden Markov Models



$$p(x_1 \dots x_n, y_1 \dots y_n) = q(STOP|y_n) \prod_{i=1}^n q(y_i|y_{i-1})e(x_i|y_i)$$

Markov Models

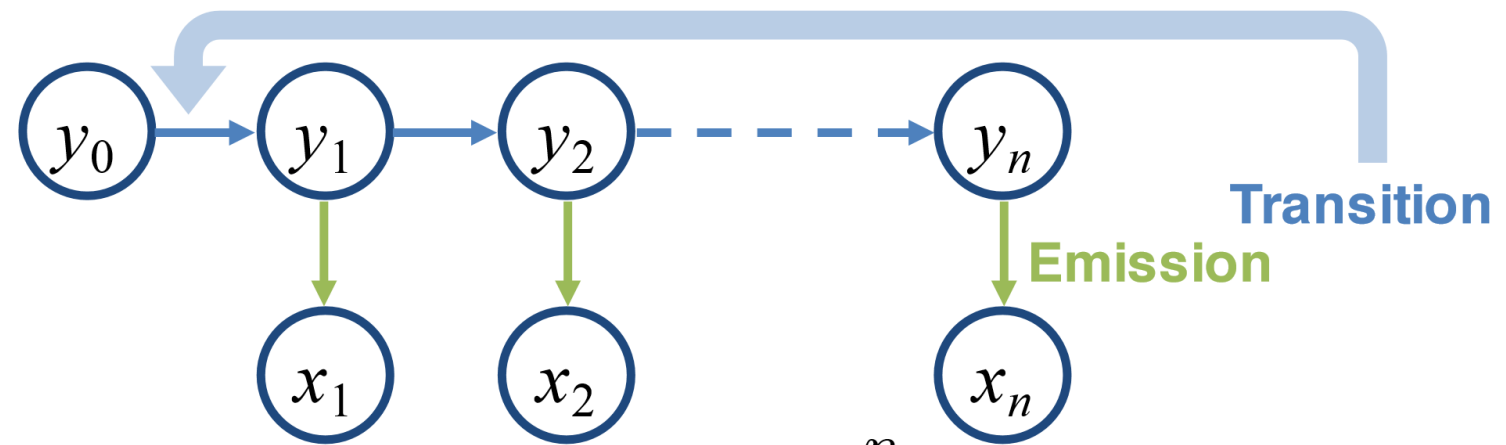
- Recall: Markov models assume the probability of a sequence can be given as a product of the *transition probabilities*

$$\Pr(x_1 x_2 \dots x_n) = \prod_i \Pr(x_i | x_{i-1})$$

- For brevity of notation, we sometimes assume that x_0 is a fixed START symbol
- Equivalent to a finite state machine with probabilistic state transitions

Hidden Markov Model

- In a hidden Markov model there are two sequences. One is a Markov chain (the y_i 's) and one is *emitted* from the Markov chain (x_i 's)
- A bi-gram (1st order Markov model) is:



$$p(x_1 \dots x_n, y_1 \dots y_n) = q(STOP|y_n) \prod_{i=1}^n q(y_i|y_{i-1})e(x_i|y_i)$$

Hidden Markov Model

- Just like with Markov language models, HMM can be of a higher-order
- For example, a 2nd order (tri-gram) HMM is

Hidden Markov Model

- Two important modeling assumptions:
 - The word (emitted value) x_i is independent of the rest of the variables given y_i
 - The POS sequence is Markovian, so a POS tag is independent of past tags given the previous tag
- POS tags don't meet either assumption
 - POS tags have dependencies that cannot be bounded in distance
 - The verb *give* usually takes two arguments, where the first can be arbitrarily long
 - Words are dependent on other words in the sentence, even given their POS
 - If you heard the verb *elapsed*, then it is likely the word *time*, *minutes*, *seconds* or *hours* also appeared in the sentence
- **Still, HMMs are a practical option**

Hidden Markov Models (HMMs)

- ▶ We have an input sentence $x = x_1, x_2, \dots, x_n$
(x_i is the i 'th word in the sentence)
- ▶ We have a tag sequence $y = y_1, y_2, \dots, y_n$
(y_i is the i 'th tag in the sentence)
- ▶ We'll use an HMM to define

$$p(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n)$$

for any sentence $x_1 \dots x_n$ and tag sequence $y_1 \dots y_n$ of the same length.

- ▶ Then the most likely tag sequence for x is

$$\arg \max_{y_1 \dots y_n} p(x_1 \dots x_n, y_1, y_2, \dots, y_n)$$

Trigram HMM

For any sentence $x_1 \dots x_n$ where $x_i \in \mathcal{V}$ for $i = 1 \dots n$, and any tag sequence $y_1 \dots y_{n+1}$ where $y_i \in \mathcal{S}$ for $i = 1 \dots n$, and $y_{n+1} = \text{STOP}$, the joint probability of the sentence and tag sequence is

$$p(x_1 \dots x_n, y_1 \dots y_{n+1}) = \prod_{i=1}^{n+1} q(y_i | y_{i-2}, y_{i-1}) \prod_{i=1}^n e(x_i | y_i)$$

where we have assumed that $x_0 = x_{-1} = *$.

Trigram HMM

For any sentence $x_1 \dots x_n$ where $x_i \in \mathcal{V}$ for $i = 1 \dots n$, and any tag sequence $y_1 \dots y_{n+1}$ where $y_i \in \mathcal{S}$ for $i = 1 \dots n$, and $y_{n+1} = \text{STOP}$, the joint probability of the sentence and tag sequence is

$$p(x_1 \dots x_n, y_1 \dots y_{n+1}) = \prod_{i=1}^{n+1} q(y_i | y_{i-2}, y_{i-1}) \prod_{i=1}^n e(x_i | y_i)$$

where we have assumed that $x_0 = x_{-1} = *$.

Parameters of the model:

- ▶ $q(s|u, v)$ for any $s \in \mathcal{S} \cup \{\text{STOP}\}$, $u, v \in \mathcal{S} \cup \{*\}$
- ▶ $e(x|s)$ for any $s \in \mathcal{S}$, $x \in \mathcal{V}$

Trigram HMM

An Example

If we have $n = 3$, $x_1 \dots x_3$ equal to the sentence *the dog laughs*, and $y_1 \dots y_4$ equal to the tag sequence D N V STOP, then

$$p(x_1 \dots x_n, y_1 \dots y_{n+1})$$

= ?

Trigram HMM

An Example

If we have $n = 3$, $x_1 \dots x_3$ equal to the sentence *the dog laughs*, and $y_1 \dots y_4$ equal to the tag sequence D N V STOP, then

$$\begin{aligned} & p(x_1 \dots x_n, y_1 \dots y_{n+1}) \\ = & q(D|*, *) \times q(N|*, D) \times q(V|D, N) \times q(\text{STOP}|N, V) \\ & \times e(\text{the}|D) \times e(\text{dog}|N) \times e(\text{laughs}|V) \end{aligned}$$

- ▶ STOP is a special tag that terminates the sequence
- ▶ We take $y_0 = y_{-1} = *$, where $*$ is a special “padding” symbol

MLE in Trigram HMMs

- Given training data in the form of sequences of words and corresponding sequences of labels, we can estimate the transition (q) and emission (e) probability by relative frequencies:

$$\hat{q}(y_3|y_2, y_1) = \frac{\#(y_1, y_2, y_3)}{\sum_y \#(y_1, y_2, y)}$$

$$\hat{e}(x_1|y_1) = \frac{\#(x_1, y_1)}{\sum_x \#(x, y_1)}$$

- Smoothing techniques are applicable

Trigram HMM

Smoothed Estimation

The transition parameters can be smoothed just like n-gram models

$$q(Vt \mid DT, JJ) = \lambda_1 \times \frac{\text{Count}(Dt, JJ, Vt)}{\text{Count}(Dt, JJ)} \\ + \lambda_2 \times \frac{\text{Count}(JJ, Vt)}{\text{Count}(JJ)} \\ + \lambda_3 \times \frac{\text{Count}(Vt)}{\text{Count}()}$$

$$\lambda_1 + \lambda_2 + \lambda_3 = 1, \quad \text{and for all } i, \lambda_i \geq 0$$

For emission probabilities we usually use pseudowords

$$e(\text{base} \mid Vt) = \frac{\text{Count}(Vt, \text{base})}{\text{Count}(Vt)}$$

HMM: Smoothing with Pseudowords

Pseudowords use our existing information about the task to categorize words that appeared very few times, or not at all, in the training data

A common method is as follows:

- ▶ **Step 1:** Split vocabulary into two sets

Frequent words = words occurring ≥ 5 times in training

Low frequency words = all other words

- ▶ **Step 2:** Map low frequency words into a small, finite set, depending on prefixes, suffixes etc.

HMM: Smoothing with Pseudowords

[Bikel et. al 1999] (named-entity recognition)

Word class	Example	Intuition
twoDigitNum	90	Two digit year
fourDigitNum	1990	Four digit year
containsDigitAndAlpha	A8956-67	Product code
containsDigitAndDash	09-96	Date
containsDigitAndSlash	11/9/89	Date
containsDigitAndComma	23,000.00	Monetary amount
containsDigitAndPeriod	1.00	Monetary amount, percentage
othernum	456789	Other number
allCaps	BBN	Organization
capPeriod	M.	Person name initial
firstWord	first word of sentence	no useful capitalization information
initCap	Sally	Capitalized word
lowercase	can	Uncapitalized word
other	,	Punctuation marks, all other words

HMM: Smoothing with Pseudowords

Profits/NA soared/NA at/NA Boeing/SC Co./CC ,/NA easily/NA
topping/NA forecasts/NA on/NA Wall/SL Street/CL ,/NA as/NA their/NA
CEO/NA Alan/SP Mulally/CP announced/NA first/NA quarter/NA
results/NA ./NA



firstword/NA soared/NA at/NA initCap/SC Co./CC ,/NA easily/NA
lowercase/NA forecasts/NA on/NA initCap/SL Street/CL ,/NA as/NA
their/NA CEO/NA Alan/SP initCap/CP announced/NA first/NA
quarter/NA results/NA ./NA

NA = No entity
SC = Start Company
CC = Continue Company
SL = Start Location
CL = Continue Location

HMM: Inference

Brute Force Search is Hopelessly Inefficient

Problem: for an input $x_1 \dots x_n$, find

$$\arg \max_{y_1 \dots y_{n+1}} p(x_1 \dots x_n, y_1 \dots y_{n+1})$$

where the $\arg \max$ is taken over all sequences $y_1 \dots y_{n+1}$ such that $y_i \in \mathcal{S}$ for $i = 1 \dots n$, and $y_{n+1} = \text{STOP}$.

HMM: Pros and Cons

- Pros:
 - A simple and widely used model with competitive performance in many sequence labeling tasks
 - Very simple to train
 - Inference is efficient
- Cons:
 - Strong independence assumption
 - Inability to use features to model the emission distribution